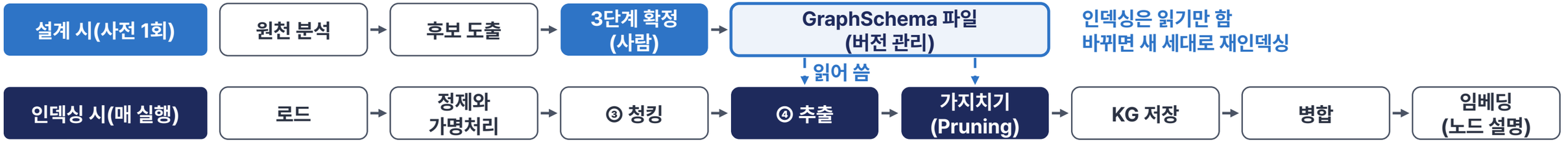
1. 스키마 도출: 노드 타입과 관계는 추출 전에 먼저 정함

스키마는 추출 결과가 아니라 추출기에 주는 허용 목록 — 먼저 정하지 않으면 같은 개념이 여러 라벨로 쪼개짐



스키마 없이 추출하면 (실측)

로컬 gemma-4 12B로 청크 2개를 스키마 없이 추출 → 라벨 11종

Brand · CardProduct = 카드 하나가 라벨 2개로

Fee · AnnualFee = 요금도 라벨 2개로

Channel · Benefit · Guardian · Grade · Date

Store · PerformanceCondition

GraphRAG 도구 기본 노드 타입/관계가 없거나 너무 일반적임

- Neo4j: 스키마를 안 주면 매번 입력 글에서 자동 추출(사람 검토 없음)
- MS GraphRAG 기본 타입: organization, person, geo, event
- LangChain+Neo4j: 허용 목록이 비면 아무 타입이나 만듦

스키마를 주면 (Native GraphSchema)

```
schema = GraphSchema.model_validate({
    "node_types": ["Card", "Benefit", "Fee"],
    "relationship_types": ["HAS_BENEFIT", "HAS_FEE"],
    "patterns": [("Card", "HAS_BENEFIT", "Benefit"),
                 ("Card", "HAS_FEE", "Fee")]})
graph = await extractor.run(chunks=chunks, schema=schema)
result = await GraphPruning().run(graph, schema)
```

① 추출 안내
허용 목록이 추출 프롬프트에 들어가 LLM이 그 안에서 고름

② 가지치기
목록 밖 노드와 관계를 지우고 지운 개수를 통계로 남김

실행 확인: 노드 타입을 주면 목록 밖 타입은 기본 불허

결론: 스키마는 인덱싱 전 설계 단계에서 확정해 파일로 고정, 인덱싱은 읽어 쓰기만 함.
정형 테이블은 변환 규칙으로, 문서는 표본 → 후보 → 사람 확정으로 스키마 설계

무단전재 및 배포 금지

1. 스키마 도출: 스키마에 정의하는 7가지

노드 타입과 관계, 각각의 속성, 연결 패턴이 뼈대 — 제약과 허용 범위로 무엇을 지킬지 정함

뼈대 ① ~ ⑤ 왼쪽부터 정의할 것, GraphSchema 칸, 카드 예

①	**노드 타입**	node_types[].label	**Card, Benefit**
②	**노드 속성**	node_types[] .properties	**Card {name, annual_fee}**
③	**관계**	relationship_types[] .label	**HAS_BENEFIT**
④	**관계 속성**	relationship_types[] .properties	**HAS_BENEFIT {valid_until}**
⑤	**연결 패턴**	patterns (출발, 관계, 도착)	**(Card, HAS_BENEFIT, Benefit)**

지킬 규칙 ⑥ ⑦

⑥	**제약**	constraints UNIQUENESS, EXISTENCE, KEY	**Card.name = KEY**
⑦	**허용 범위**	additional_node_types, additional_relationship_types, additional_patterns	**①~⑤에 지정된 목록 밖을 허용할지 여부**



7칸을 채운 Native GraphSchema — 실행 확인: name 없는 Card는 가지치기가 지움

```
schema = GraphSchema.model_validate({
  "node_types": [                          # ① ②
    {"label": "Card", "properties": [
       {"name": "name", "type": "STRING"},
       {"name": "annual_fee", "type": "INTEGER"}]},
    {"label": "Benefit", "properties": [...]}],
  "relationship_types": [{"label": "HAS_BENEFIT",   # ③ ④
     "properties": [{"name": "valid_until", "type": "DATE"}]}],
  "patterns": [("Card", "HAS_BENEFIT", "Benefit")],   # ⑤
  "constraints": [{"type": "KEY", "node_type": "Card",  # ⑥
                   "property_names": ["name"]}],
  "additional_node_types": False})              # ⑦
```

※ 최소한 = ①, ③, ⑤ + ⑥ 고유키. 속성은 질문에 쓰는 것만 정의하고, 타입마다 description을 적으면 LLM이 고르는 기준이 됨
※ 제약: 지정한 속성에 대해 UNIQUENESS는 고유 강제, EXISTENCE는 존재 강제, KEY는 고유 + 존재를 강제함. UNIQUENESS는 지정한 속성이 비어 있어도 허용하나 KEY는 불허함

무단전재 및 배포 금지

1. 스키마 도출: 정형 테이블 → 노드 타입과 관계: 변환 규칙

모델 없이 규칙으로 바꿈 — 단, FK를 세기 전에 '그 표 자체가 조회 대상인가'부터 물음

테이블 1개

Q1. 그 표 자체가 조회 대상인 엔티티인가?

예

① 노드 타입
열 = 노드 속성
FK = 관계
FK 열은 바꾼 뒤 지움

FK가 0개 또는 1개인
보통 테이블은 대부분 여기로 옴

아니오

Q2. FK 몇 개?

② 조인 테이블: 2개이고 업무 열 없음
관계 1개로 흡수

③ 조인 테이블: 2개 + 업무 속성
관계 + 관계 속성
(조회하거나 집계하면 노드)

④ 3개 이상
노드 타입(중간 노드)

왜 Q1이 먼저? FK 수만 보면 이체 기록(FK 2개)이 관계로 흡수돼 이체 자체를 찾을 수 없음 (예 ①)

① 이체(출금계좌FK, 입금계좌FK, 금액, 일시) → **:계좌** —:출금→ **:이체** —:입금→ **:계좌**
조회 대상 → 노드 타입(FK가 2개여도)

② 사용자, 역할, 사용자역할(사용자FK, 역할FK) → **:사용자** —:맡음→ **:역할**
업무 열 없음 → 관계로 흡수(사용자역할 노드 없음)

③ 주문상품(주문FK, 상품FK, 수량, 단가) → **:주문** —:포함 {수량, 단가}→ **:상품**
업무 속성 → 관계 속성(판매 집계가 잦으면 노드)

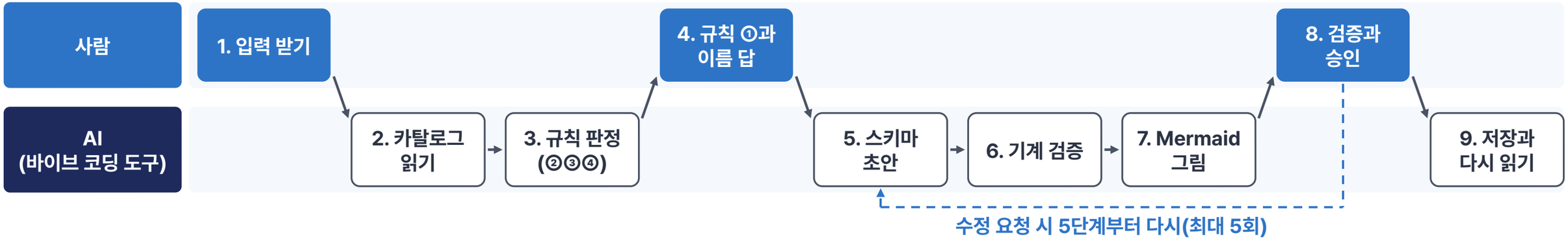
④ 수강신청(학생FK, 과목FK, 학기FK) → **:학생** —:신청→ **:수강신청** —:대상→ **:과목**
FK 3개 → 중간 노드(학기도 같은 방식), 홉 +1

Neo4j 공식 변환 규칙과 같음: 테이블 → 라벨 , FK → 관계 , 조인 테이블 → 관계 또는 관계 + 속성, 셋 이상 → 중간 노드
이름에 공백 없음(Neo4j 권고: 라벨은 PascalCase, 관계는 대문자_밑줄). 정형에서 나온 타입은 LLM 추출 비용이 들지 않음

무단전재 및 배포 금지

1. 스키마 도출: 프롬프트로 정형 테이블의 스키마 도출

변환 규칙을 프롬프트에 담아 바이브 코딩 도구에 맡김 — AI는 읽고 판정하고 그리며, 사람은 세 번 답하고 승인함



사람이 답하는 곳 3번

1 출력 폴더와 DB 접속 정보
비밀번호는 .env에 직접 등록

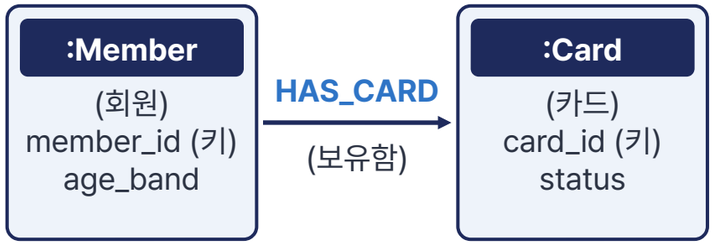
4 규칙 ① 후보(FK 2개 + 업무 열)를 노드 타입 또는 관계 + 관계 속성으로 정함
뺄 테이블, 이름 후보 승인

8 Mermaid 그림과 판정표를 보고 승인
(승인자와 일시를 기록)

AI가 지키는 규칙

- FK 개수는 열 수가 아니라 제약 수로 셈
- DB는 읽기 전용, 비밀번호는 .env에만 둠
- 라벨은 영문, 한글 이름은 description에 넣음
- 그림은 Mermaid만 씀
- 규칙 ① 판정과 이름은 추측해 확정하지 않음
- 후보 표시가 남으면 스키마 파일을 만들지 않음

승인 화면 예 (Mermaid 그림)



영문 라벨 아래에 한글 이름을 함께 적어
검토하는 사람이 바로 읽게 함

결과: graph_schema.json(승인본)과 approval.json(승인자, 승인 일시, 입력 파일 SHA-256). 인덱싱은 이 파일을 읽기만 함
GraphSchema 검증, 저장, 다시 읽기는 Neo4j 서버 없이 됨(실행 확인)

무단전재 및 배포 금지

1. 스키마 도출: 소스 문서 → 노드 타입과 관계: 표본 → 후보 → 사람 확정

LLM은 표본에서 후보만 뽑고 확정은 사람이 함 — 확정한 목록을 GraphSchema로 고정해 추출기에 넘김

문서 종류별 표본 N건 → 스키마 추출기 → 후보 → 정형 후보와 합침 → 3단계 확정 (사람) → GraphSchema 고정 → 추출기

후보 뽑는 도구

Native neo4j-graphrag	후보: SchemaFromTextExtractor().run(text=표본) 확정: 고친 스키마를 schema로 넘김(안 넘기면 매번 자동)
Neo4j LLM KG Builder	후보: 'Get Schema From Text'가 스키마 제안 확정: 화면에서 사람이 고름
MS GraphRAG	후보: graphrag prompt-tune --discover-entity-types 확정: 저장된 프롬프트 파일을 사람이 고침
LangChain과 LlamaIndex	후보: 자동 후보 단계 없음 확정: 사람이 허용 목록을 먼저 적음

3단계 확정 — 사람이 줄이고 이름을 정함

1 같은 의미는 합치고 이름을 정함

온톨로지: 용어 나열과 계층 정리

예: Brand와 CardProduct → Card, Fee와 AnnualFee → Fee

2 쓰는 워크플로우 단계가 없는 관계는 뺌

온톨로지: 역량 질문(답해야 할 질문이 범위를 정함)

예: 어느 단계도 따라가지 않는 관계 → 다음 회차 후보로 1줄

3 적재 원천이 없는 노드 타입은 뺌

가이드 고유 규칙(빈 노드 방지)

예: 어느 원천도 채우지 않는 타입 → 정의하지 않음

! 표본에 없던 타입은 안 나옴
→ 문서 종류마다 표본, 표본 수 기록

! 도구 기본값(organization, person, geo, event) 그대로 금지

! 빼면 그래프에 아예 없음
(가리는 것은 접근 필터의 일)

! 확정 뒤 역량 질문으로 되짚기
(가이드에 없는 보완점)

무단전재 및 배포 금지